

Analizador Sintático (Parser)

Visão Geral

O parser recebe a sequência de tokens do lexer e produz uma Árvore Sintática Abstracta (AST). É implementado com `PLY yacc (LALR(1))` e organizado em dois ficheiros:

- `parser.py` — gramática, produções `PLY`, tratamento de erros, `ASTVisitor`, `ASTPrinter`
- `ast.py` — hierarquia de nós da AST

Gramática Implementada

A gramática segue de perto a especificação formal do projecto, com algumas adaptações necessárias para a integração com `PLY`.

Estrutura do programa

```
program          → program_unit  
                  | program program_unit  
  
program_unit     → main_program  
                  | function_subprogram  
                  | subroutine_subprogram  
  
main_program     → PROGRAM IDEN NEWLINE body END NEWLINE
```

Subprogramas

```
function_subprogram → type_spec FUNCTION IDEN '(' param_list ')' NEWLINE  
body END NEWLINE  
                   | FUNCTION IDEN '(' param_list ')' NEWLINE body END  
NEWLINE  
                   | type_spec FUNCTION IDEN '(' ')' NEWLINE body END  
NEWLINE  
                   | FUNCTION IDEN '(' ')' NEWLINE body END NEWLINE  
  
subroutine_subprogram → SUBROUTINE IDEN '(' param_list ')' NEWLINE body END  
NEWLINE  
                     | SUBROUTINE IDEN '(' ')' NEWLINE body END NEWLINE  
  
param_list          → IDEN  
                     | param_list ',' IDEN
```

As quatro variantes de `function_subprogram` cobrem todas as combinações de tipo de retorno explícito/implícito e lista de parâmetros vazia/não-vazia. Esta expansão evita reduções ambíguas no

autômato LALR(1).

Body, declarações e instruções

```

body          → decl_section stmt_section

decl_section  → ε
               | decl_section declaration NEWLINE

stmt_section  → ε
               | stmt_section stmt_line

stmt_line     → statement NEWLINE
               | LABEL statement NEWLINE
               | LABEL CONTINUE NEWLINE

declaration   → type_spec var_decl_list

type_spec     → INTEGER | REAL | LOGICAL | CHARACTER
               | CHARACTER '*' INT_LIT

var_decl_list → var_decl
               | var_decl_list ',' var_decl

var_decl      → IDEN
               | IDEN '(' INT_LIT ')'
  
```

A produção **LABEL CONTINUE NEWLINE** foi separada de **LABEL statement NEWLINE** porque **CONTINUE** é um terminador de DO loops — tê-lo como produção própria simplifica o reconhecimento sem ambiguidade.

Instruções executáveis

```

statement     → assignment_stmt | if_stmt | do_stmt | goto_stmt
               | print_stmt | read_stmt | call_stmt | return_stmt |
               stop_stmt

assignment_stmt → variable '=' expr

if_then_block  → IF '(' expr ')' THEN NEWLINE stmt_section ENDIF
               | IF '(' expr ')' THEN NEWLINE stmt_section ELSE NEWLINE
               stmt_section ENDIF

logical_if_stmt → IF '(' expr ')' statement

do_stmt       → DO INT_LIT IDEN '=' expr ',' expr NEWLINE stmt_section
               LABEL CONTINUE
               | DO INT_LIT IDEN '=' expr ',' expr ',' expr NEWLINE
               stmt_section LABEL CONTINUE
  
```

```
goto_stmt      → GOTO INT_LIT
print_stmt     → PRINT '*' ',' io_list
read_stmt      → READ '*' ',' io_list
call_stmt      → CALL IDEN '(' expr_list ')' | CALL IDEN '(' ')'
return_stmt    → RETURN
stop_stmt      → STOP
```

Expressões

```
expr           → expr arithmetic_op expr
               | expr logical_op expr
               | expr relational_op expr
               | unary_arithmetic_op expr
               | unary_logical_op expr
               | '(' expr ')'
               | INT_LIT | REAL_LIT | STRING_LIT | TRUE | FALSE
               | variable | func_call

variable       → IDEN
               | IDEN '(' expr_list ')'
               | IDEN '(' ')'

func_call      → MOD '(' expr ',' expr ')'
               | SQRT '(' expr ')'
               | MAX '(' expr_list ')'
               | MIN '(' expr_list ')'
```

Tabela de Precedência

A tabela de precedência do PLY resolve os conflitos shift-reduce em expressões:

```
precedence = (
    ('left',      'OP_OR'),
    ('left',      'OP_AND'),
    ('right',     'OP_NOT'),
    ('nonassoc',  'OP_EQ', 'OP_NE', 'OP_LT', 'OP_LE', 'OP_GT', 'OP_GE'),
    ('left',      '+', '-'),
    ('left',      '*', '/'),
    ('right',     'UMINUS', 'UPLUS'),
    ('right',     'POWER'),
    ('right',     'NEWLINE'),
    ('right',     'DO_TERM'),
)
```

Da menor para a maior precedência:

1. **OR** (esquerda)

2. **AND** (esquerda)
3. **NOT** (unário direita)
4. Operadores relacionais (**nonassoc** — não associativos, como em Fortran)
5. Adição/subtração (esquerda)
6. Multiplicação/divisão (esquerda)
7. **UMINUS / UPLUS** — pseudotokens para resolver o conflito unário/binário de **+** e **-**
8. **POWER** (******, direita) — a potência é associativa à direita em Fortran
9. **NEWLINE** e **DO_TERM** — com maior precedência para resolver o conflito do **DO** loop

Pseudotokens UMINUS, UPLUS, DO_TERM

São tokens que nunca são produzidos pelo lexer; existem apenas para atribuir precedência a produções específicas via **%prec**:

```
# Unary minus/plus
def p_expr_arithmetic_unary(self, p):
    '''expr : '-' expr %prec UMINUS
           | '+' expr %prec UPLUS'''
    p[0] = ArithmeticUnaryOp(op=p[1], operand=p[2], lineno=p.lineno(1))

# DO loop – %prec DO_TERM evita que o LABEL CONTINUE dentro do body seja
# consumido prematuramente antes de o parser reduzir a produção do DO
def p_do_loop(self, p):
    """do_stmt : DO INT_LIT IDEN '=' expr ',' expr NEWLINE stmt_section
               LABEL CONTINUE %prec DO_TERM"""
```

O **DO_TERM** foi necessário porque o parser LALR(1) pode ter dificuldade em distinguir, dentro de **stmt_section**, se um **LABEL CONTINUE** é parte do body do DO ou o terminador. A directiva **%prec DO_TERM** com a mais alta precedência força a redução correcta.

Árvore Sintática Abstracta (AST)

Classe base **Node**

```
@dataclass
class Node:
    lineno: int

    def accept(self, visitor):
        method = 'visit_' + type(self).__name__
        visit = getattr(visitor, method, visitor.generic_visit)
        return visit(self)
```

Todos os nós são **dataclass** que herdam de **Node**. O método **accept()** implementa o padrão Visitor por reflexão: procura o método **visit_<NomeDoNó>** no visitante; se não existir, usa **generic_visit**. Isto

desacopla completamente a AST dos algoritmos que a percorrem (análise semântica, geração de código, etc.).

Hierarquia de nós

Estrutura do programa

Nó	Campos principais	Descrição
Program	units: list[Node]	Raiz; contém todas as unidades
MainProgram	name, body	PROGRAM NAME ... END
FunctionDef	name, params, return_type, body	[TYPE] FUNCTION NAME(...)
SubroutineDef	name, params, body	SUBROUTINE NAME(...)
Body	declarations, statements	Secção de declarações + instruções

Declarações

Nó	Campos	Descrição
TypeDecl	type_name, char_len, variables	INTEGER N, NUMS(5)
VarDecl	name, dimension	Um item na lista: escalar (dim=0) ou array (dim>=1)

`char_len` em `TypeDecl` é usado apenas para `CHARACTER*N`; é `None` para os outros tipos.

Instruções executáveis

Nó	Campos	Descrição
LabeledStmt	label, stmt	Qualquer instrução com label numérico
Assign	target, value	<code>VAR = expr</code> ou <code>ARR(i) = expr</code>
IfThen	condition, then_body, else_body	IF-THEN-[ELSE]-ENDIF
LogicalIf	condition, stmt	IF (cond) stmt (sem THEN)
DoLoop	label, var, start, stop, step, body	DO; step=None → incremento 1
Goto	label	GOTO N
Continue	—	CONTINUE standalone
PrintStmt	items	PRINT *, lista
ReadStmt	targets	READ *, lista

Nó	Campos	Descrição
CallStmt	name, args	CALL SUB(args)
ReturnStmt	—	RETURN
StopStmt	—	STOP

Expressões

Nó	Campos	Descrição
ArithmeticBinOp	left, op, right	+, -, *, /, **
LogicalBinOp	left, op, right	.AND., .OR.
RelationalBinOp	left, op, right	.EQ., .NE., .LT., .LE., .GT., .GE.
ArithmeticUnaryOp	op, operand	-expr, +expr
LogicalUnaryOp	op, operand	.NOT. expr
FuncCall	name, args	Funções intrínsecas: MOD, SQRT, MAX, MIN
Var	name	Variável simples
VarOrFuncCall	name, args	Array A(i) ou chamada de função F(x)
IntLit	value: int	Literal inteiro
RealLit	value: float	Literal real
StringLit	value: str	Literal de string
BoolLit	value: bool	.TRUE. ou .FALSE.

Nó VarOrFuncCall — Ambiguidade sintática intencional

No Fortran 77, a sintaxe NAME(args) é ambígua a nível sintático: pode ser um acesso a array ou uma chamada de função. Esta distinção só é resolvível com a tabela de símbolos, que ainda não existe durante o parsing. A decisão foi criar o nó VarOrFuncCall para representar ambos os casos, deixando a desambiguação para a análise semântica.

As funções intrínsecas (MOD, SQRT, MAX, MIN) têm tokens próprios e produções gramaticais dedicadas, sendo sempre representadas por FuncCall — nunca por VarOrFuncCall.

Padrão Visitor

ASTVisitor

```
class ASTVisitor:
    def visit(self, node: Node):
        return node.accept(self)
```

```
def generic_visit(self, node: Node):
    for child in self._children(node):
        if isinstance(child, Node):
            self.visit(child)
        elif isinstance(child, list):
            for item in child:
                if isinstance(item, Node):
                    self.visit(item)

def _children(self, node: Node):
    from dataclasses import fields
    return [getattr(node, f.name) for f in fields(node)]
```

O `generic_visit` percorre todos os campos do dataclass por reflexão. Visitantes que só precisam de tratar alguns nós não precisam de implementar os restantes — o `generic_visit` desce automaticamente a árvore.

ASTPrinter

Visitante de debug incluído no módulo. Imprime a AST com indentação, mostrando o tipo de cada nó e o número de linha. Invocado com a flag `-p` na linha de comandos.

Tratamento de Erros Sintáticos

```
def p_error(self, p):
    if p is None:
        self._errors.append(ParserError(message="Fim de ficheiro inesperado", line=0))
    else:
        self._errors.append(ParserError(
            message=f"Token inesperado '{p.value}' (tipo: {p.type})",
            line=p.lineno,
        ))
        # Recuperação: descarta tokens até ao próximo NEWLINE
        while True:
            tok = self._parser.token()
            if tok is None or tok.type == 'NEWLINE':
                break
        self._parser.errok()
```

A estratégia de recuperação de erro é **panic mode por linha**: ao encontrar um token inesperado, o parser descarta todos os tokens até ao próximo `NEWLINE` e chama `errok()` para sair do modo de erro do PLY. Esta abordagem permite reportar múltiplos erros sintáticos em diferentes linhas de uma única passagem, em vez de parar no primeiro erro.

Integração com o Lexer

O parser instancia internamente um `Lexer` em cada chamada a `parse()`:

```
def parse(self, source: str) -> Program | None:
    self._errors = []
    lexer = Lexer()
    result = self._parser.parse(input=source, lexer=lexer)
    # propaga erros léxicos
    for err in lexer.errors:
        self._errors.append(ParserError(message=f"[Léxico] {err.message}",
line=err.line))
    return result
```

Os erros léxicos são propagados para a lista de erros do parser, garantindo que a interface externa só precisa de verificar `parser.errors` para obter todos os erros da fase de análise léxico-sintática.